

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

***CO1:** Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

QUESTIONS: 10 Total Marks: 50 Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement epsilon-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using epsilon-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Code of Conduct:

I A Afran / 192325024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION

Criteria		Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Experiment 1: Installation of Reinforcement Learning Development Environment

Problem Statement

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Planning

- Set up a dedicated Anaconda environment and install NumPy, Matplotlib, Gymnasium, and TensorFlow/Keras.
- Write a verification script that programmatically checks each required package is importable and prints its installed version, rather than assuming installation succeeded silently.
- As a functional sanity check (not just an import check), run a random agent for 50 steps in CartPole-v1 to confirm the RL environment stack (Gymnasium + NumPy) works end-to-end.
- Additionally check whether TensorFlow was built with CUDA support and whether it detects a GPU, printing a clear message either way so the environment report reflects the actual hardware being used.
- Treat this experiment as a gate: only proceed to later experiments once every package reports [OK].

Pseudocode

```
BEGIN
DEFINE required_packages = [NumPy, Matplotlib, Gymnasium, TensorFlow, Keras]
all_ok <- True
FOR each package in required_packages:
TRY import package ; PRINT "[OK]" with version
CATCH ImportError: PRINT "[MISSING]" ; all_ok <- False
IF NOT all_ok:
PRINT install instructions ; EXIT
// Functional sanity check
env <- gym.make("CartPole-v1")
reset env with seed=42
FOR step in 1..50:
action <- env.action_space.sample() // random action
step env, accumulate total_reward
IF episode ended: reset env
PRINT total_reward collected
PRINT whether TensorFlow was built with CUDA and whether a GPU is detected
PRINT "Environment verified successfully!"
END
```

Code (Python)

```
"""
Experiment 1: Installation of Reinforcement Learning Development Environment
-----
Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras,
Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and
verify the environment by executing a simple Reinforcement Learning program.
"""

import sys
import importlib

def check_package(display_name, import_name=None):
    import_name = import_name or display_name
    try:
        module = importlib.import_module(import_name)
        version = getattr(module, "__version__", "unknown")
        print(f"[OK] {display_name:12s} -> version {version}")
        return True
    except ImportError as e:
        print(f"[MISSING] {display_name:12s} -> {e}")
        return False

def main():
    print("=" * 60)
    print("Experiment 1: RL Development Environment Verification")
    print("=" * 60)
```

```

print(f"Python version: {sys.version.split()[0]}\n")

packages = [
    ("NumPy", "numpy"),
    ("Matplotlib", "matplotlib"),
    ("Gymnasium", "gymnasium"),
    ("TensorFlow", "tensorflow"),
    ("Keras", "keras"),
]

all_ok = True
for display_name, import_name in packages:
    ok = check_package(display_name, import_name)
    all_ok = all_ok and ok

print("\n" + "=" * 60)
if not all_ok:
    print("Some packages are missing. Install them first, e.g.:")
    print(' pip install numpy matplotlib gymnasium[classic-control,toy-text] tensorflow')
    print("=" * 60)
    return

print("All required packages are installed. Running a sanity-check program...\n")

# --- Simple RL sanity check: random agent on CartPole ---
import gymnasium as gym

env = gym.make("CartPole-v1")
obs, info = env.reset(seed=42)
total_reward = 0
for _ in range(50):
    action = env.action_space.sample() # random action
    obs, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    if terminated or truncated:
        obs, info = env.reset()
env.close()
print(f"Ran 50 random steps in CartPole-v1. Total reward collected: {total_reward}")

# --- TensorFlow / GPU check ---
import tensorflow as tf

print(f"\nTensorFlow built with CUDA support: {tf.test.is_built_with_cuda()}")
gpus = tf.config.list_physical_devices("GPU")
if gpus:
    print(f"GPU(s) detected by TensorFlow: {gpus}")
else:
    print("No GPU detected by TensorFlow (running on CPU). This is fine for all")
    print("experiments in this lab -- the networks used are small.")

print("\nEnvironment verified successfully!")
print("=" * 60)

if __name__ == "__main__":
    main()

```

Environment / Dataset Used

Local Anaconda/Python environment (NumPy, Matplotlib, Gymnasium, TensorFlow, Keras) verified against Gymnasium's CartPole-v1 environment. No external dataset required.

Output

```

=====
Experiment 1: RL Development Environment Verification
=====
Python version: 3.12.3
[OK] NumPy -> version 2.4.4
[OK] Matplotlib -> version 3.10.8
[OK] Gymnasium -> version 1.3.0
[OK] TensorFlow -> version 2.21.0

```

```
[OK] Keras -> version 3.15.0
```

```
=====
All required packages are installed. Running a sanity-check program...
```

```
Ran 50 random steps in CartPole-v1. Total reward collected: 50.0
```

```
TensorFlow built with CUDA support: False
No GPU detected by TensorFlow (running on CPU). This is fine for all
experiments in this lab -- the networks used are small.
```

```
Environment verified successfully!
=====
```

All five required packages (NumPy, Matplotlib, Gymnasium, TensorFlow, Keras) were detected and imported successfully. The functional check ran 50 random steps in CartPole-v1 without error (total reward collected: 50.0), and TensorFlow correctly reported no GPU/CUDA on this machine, confirming the environment is ready for the rest of the lab.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Problem Statement

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Planning

- Create each of the three standard environments (FrozenLake-v1, CartPole-v1, MountainCar-v0) via `gym.make()`, and print their `observation_space` and `action_space` to see how state and action representations differ across environment types (discrete vs continuous).
- Reset each environment with a fixed seed (42) to get a reproducible initial state.
- Step through each environment for 10 steps using random actions (`env.action_space.sample()`), printing the action taken, resulting next state, reward, and termination/truncation flags at every step, to make the state-action-reward-episode loop concrete.
- Handle episode termination explicitly: if an episode ends mid-run (e.g., FrozenLake falling in a hole), reset the environment and continue stepping, demonstrating the episode boundary concept.
- Compare the three environments side-by-side: FrozenLake has a small discrete state space (16 states) and can terminate quickly; CartPole and MountainCar have continuous (Box) observation spaces and different reward structures (CartPole rewards survival, MountainCar penalizes every step).

Pseudocode

```
BEGIN
FUNCTION explore_env(env_name, n_steps, **kwargs):
  env <- gym.make(env_name, **kwargs)
  PRINT env.observation_space, env.action_space
  obs <- env.reset(seed=42)
  PRINT initial state
  total_reward <- 0
  FOR step in 1..n_steps:
    action <- env.action_space.sample()
    obs, reward, terminated, truncated <- env.step(action)
    total_reward += reward
    PRINT step details
    IF terminated OR truncated:
      obs <- env.reset() // episode boundary
      PRINT total_reward
  env.close()

explore_env("FrozenLake-v1", 10, is_slippery=False)
explore_env("CartPole-v1", 10)
```

```
explore_env("MountainCar-v0", 10)
END
```

Code (Python)

```
"""
Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments
-----
Aim: To create, initialize, and interact with standard Reinforcement Learning
environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to
understand the concepts of states, actions, rewards, and episodes.
"""

import gymnasium as gym

def explore_env(env_name, n_steps=10, **env_kwargs):
    print(f"\n{'=' * 60}")
    print(f"Environment: {env_name}")
    print(f"{'=' * 60}")

    env = gym.make(env_name, **env_kwargs)
    print(f"Observation space: {env.observation_space}")
    print(f"Action space: {env.action_space}")

    obs, info = env.reset(seed=42)
    print(f"Initial state: {obs}")

    total_reward = 0
    for step in range(1, n_steps + 1):
        action = env.action_space.sample() # random action for demonstration
        obs, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        print(
            f"Step {step:2d}: action={action} next_state={obs} "
            f"reward={reward} terminated={terminated} truncated={truncated}"
        )
        if terminated or truncated:
            print(">> Episode finished, resetting environment.")
            obs, info = env.reset()

    print(f"Total reward collected over {n_steps} steps: {total_reward}")
    env.close()

if __name__ == "__main__":
    # FrozenLake: discrete states (0-15), discrete actions (0-3: LEFT/DOWN/RIGHT/UP)
    explore_env("FrozenLake-v1", n_steps=10, is_slippery=False)

    # CartPole: continuous state (cart pos, vel, pole angle, angular vel), 2 actions
    explore_env("CartPole-v1", n_steps=10)

    # MountainCar: continuous state (position, velocity), 3 actions
    explore_env("MountainCar-v0", n_steps=10)
```

Environment / Dataset Used

Gymnasium's built-in FrozenLake-v1, CartPole-v1, and MountainCar-v0 environments. No external dataset required.

Output

```
=====
Environment: FrozenLake-v1
=====
Observation space: Discrete(16)
Action space: Discrete(4)
Initial state: 0
Step 1: action=1 next_state=4 reward=0 terminated=False truncated=False
Step 2: action=3 next_state=0 reward=0 terminated=False truncated=False
Step 3: action=1 next_state=4 reward=0 terminated=False truncated=False
```

```

Step 4: action=1 next_state=8 reward=0 terminated=False truncated=False
Step 5: action=0 next_state=8 reward=0 terminated=False truncated=False
Step 6: action=2 next_state=9 reward=0 terminated=False truncated=False
Step 7: action=3 next_state=5 reward=0 terminated=True truncated=False
>> Episode finished, resetting environment.
Step 8: action=1 next_state=4 reward=0 terminated=False truncated=False
Step 9: action=2 next_state=5 reward=0 terminated=True truncated=False
>> Episode finished, resetting environment.
Step 10: action=1 next_state=4 reward=0 terminated=False truncated=False
Total reward collected over 10 steps: 0

```

```

=====
Environment: CartPole-v1
=====

```

```

Observation space: Box([-4.8 -inf -0.41887903 -inf], [4.8 inf 0.41887903 inf], (4,), float32)
Action space: Discrete(2)
Initial state: [ 0.0273956 -0.00611216 0.03585979 0.0197368 ]
Step 1: action=0 next_state=[ 0.02727336 -0.20172954 0.03625453 0.32351476] reward=1.0 terminated=False
truncated=False
Step 2: action=0 next_state=[ 0.02323877 -0.39734846 0.04272482 0.62740684] reward=1.0 terminated=False
truncated=False
Step 3: action=1 next_state=[ 0.0152918 -0.20284806 0.05527296 0.34847975] reward=1.0 terminated=False
truncated=False
Step 4: action=1 next_state=[ 0.01123484 -0.00855404 0.06224256 0.07372576] reward=1.0 terminated=False
truncated=False
Step 5: action=0 next_state=[ 0.01106376 -0.20451054 0.06371707 0.3853788 ] reward=1.0 terminated=False
truncated=False
Step 6: action=0 next_state=[ 0.00697355 -0.40047646 0.07142465 0.69745135] reward=1.0 terminated=False
truncated=False
Step 7: action=1 next_state=[-0.00103598 -0.2064138 0.08537367 0.42808053] reward=1.0 terminated=False
truncated=False
Step 8: action=1 next_state=[-0.00516426 -0.01259816 0.09393528 0.16348532] reward=1.0 terminated=False
truncated=False
Step 9: action=0 next_state=[-0.00541622 -0.20893054 0.09720499 0.4842619 ] reward=1.0 terminated=False
truncated=False
Step 10: action=1 next_state=[-0.00959483 -0.01530512 0.10689023 0.22372814] reward=1.0 terminated=False
truncated=False
Total reward collected over 10 steps: 10.0

```

```

=====
Environment: MountainCar-v0
=====

```

```

Observation space: Box([-1.2 -0.07], [0.6 0.07], (2,), float32)
Action space: Discrete(3)
Initial state: [-0.4452088 0. ]
Step 1: action=0 next_state=[-0.44679132 -0.00158252] reward=-1.0 terminated=False truncated=False
Step 2: action=1 next_state=[-0.4489448 -0.00215349] reward=-1.0 terminated=False truncated=False
Step 3: action=0 next_state=[-0.45265353 -0.00370873] reward=-1.0 terminated=False truncated=False
Step 4: action=0 next_state=[-0.45789033 -0.00523681] reward=-1.0 terminated=False truncated=False
Step 5: action=0 next_state=[-0.46461678 -0.00672644] reward=-1.0 terminated=False truncated=False
Step 6: action=0 next_state=[-0.47278327 -0.0081665 ] reward=-1.0 terminated=False truncated=False
Step 7: action=0 next_state=[-0.48232943 -0.00954614] reward=-1.0 terminated=False truncated=False
Step 8: action=1 next_state=[-0.49218428 -0.00985487] reward=-1.0 terminated=False truncated=False
Step 9: action=1 next_state=[-0.5022744 -0.01009013] reward=-1.0 terminated=False truncated=False
Step 10: action=0 next_state=[-0.51352435 -0.01124995] reward=-1.0 terminated=False truncated=False
Total reward collected over 10 steps: -10.0

```

FrozenLake-v1 (16 discrete states, 4 actions) terminated twice within 10 random steps (falling into holes), illustrating short, high-variance episodes. CartPole-v1 (continuous 4-D state, 2 actions) survived the full 10 steps, accumulating a reward of 10.0 as expected (+1 per surviving step). MountainCar-v0 (continuous 2-D state, 3 actions) accumulated -10.0 over 10 steps, consistent with its -1-per-step reward structure that only stops once the goal is reached.

Experiment 3: Implementation of the Reinforcement Learning Framework

Problem Statement

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment

through states, actions, rewards, and policies using Python.

Planning

- Build a minimal custom environment (GridWorldEnv) from scratch to make the agent-environment interface explicit, rather than relying on a pre-built Gymnasium environment: it exposes `reset()` and `step(action)` exactly like the standard RL loop expects.
- Represent state as an (x, y) grid position, action as one of four directions, and define the reward function directly in `step()`: -1 for every non-terminal move (encourages reaching the goal quickly), +10 for reaching the bottom-right goal cell.
- Implement a separate Agent class holding only a `policy()` function, keeping the agent decoupled from the environment -- this Agent uses a uniform random policy, deliberately the simplest possible policy so the mechanics of the loop are the focus, not the algorithm.
- Implement the standard RL interaction loop explicitly: at every timestep, query the agent's policy for an action given the current state, step the environment, accumulate reward, and check the done flag to end the episode.
- Run one detailed, verbose episode to show the full state-action-reward trace, then run five more episodes silently to show that a random policy reaches the goal inconsistently (some episodes succeed in a handful of steps, others hit the step limit without success) -- motivating why later experiments replace this random policy with a learned one.

Pseudocode

```
BEGIN
CLASS GridWorldEnv:
goal <- (size-1, size-1) ; state <- (0,0)
FUNCTION reset(): state <- (0,0) ; RETURN state
FUNCTION step(action):
move state by action, clipped to grid bounds
done <- (state == goal)
reward <- 10 if done else -1
RETURN state, reward, done

CLASS Agent:
FUNCTION policy(state): RETURN random.choice(actions) // uniform random policy

FUNCTION run_episode(env, agent, max_steps):
state <- env.reset() ; total_reward <- 0
FOR t in 1..max_steps:
action <- agent.policy(state)
next_state, reward, done <- env.step(action)
total_reward += reward ; state <- next_state
IF done: RETURN total_reward, t
RETURN total_reward, max_steps // ran out of steps

run_episode(env, agent, verbose=True) // one detailed trace
FOR ep in 1..5: run_episode(env, agent, verbose=False) // summary over 5 episodes
END
```

Code (Python)

```
"""
Experiment 3: Implementation of the Reinforcement Learning Framework
-----
Aim: To implement the basic Reinforcement Learning framework by designing an
agent that interacts with an environment through states, actions, rewards, and
policies using Python.
"""

import random
class GridWorldEnv:
    """A simple size x size grid world. Agent starts at top-left, goal is
    bottom-right. Every step costs -1, reaching the goal gives +10."""

    def __init__(self, size=4):
        self.size = size
        self.goal = (size - 1, size - 1)
        self.state = (0, 0)
        self.actions = ["UP", "DOWN", "LEFT", "RIGHT"]
```



```

def reset(self):
    self.state = (0, 0)
    return self.state

def step(self, action):
    x, y = self.state
    if action == "UP":
        x = max(0, x - 1)
    elif action == "DOWN":
        x = min(self.size - 1, x + 1)
    elif action == "LEFT":
        y = max(0, y - 1)
    elif action == "RIGHT":
        y = min(self.size - 1, y + 1)
    self.state = (x, y)

    done = self.state == self.goal
    reward = 10 if done else -1
    return self.state, reward, done

class Agent:
    """A simple agent following a uniform random policy."""

    def __init__(self, actions):
        self.actions = actions

    def policy(self, state):
        return random.choice(self.actions)

def run_episode(env, agent, max_steps=50, verbose=True):
    state = env.reset()
    total_reward = 0
    for t in range(1, max_steps + 1):
        action = agent.policy(state)
        next_state, reward, done = env.step(action)
        total_reward += reward
        if verbose:
            print(f"Step {t:2d}: state={state} action={action:5s} "
                  f"next_state={next_state} reward={reward}")
        state = next_state
        if done:
            print(f"\nGoal reached in {t} steps! Total reward: {total_reward}")
            return total_reward, t
    print(f"\nMax steps reached without finding goal. Total reward: {total_reward}")
    return total_reward, max_steps

if __name__ == "__main__":
    random.seed(7)
    env = GridWorldEnv(size=4)
    agent = Agent(env.actions)

    print("=" * 60)
    print("Single episode (random policy) -- detailed trace")
    print("=" * 60)
    run_episode(env, agent, max_steps=50, verbose=True)

    print("\n" + "=" * 60)
    print("Summary over 5 episodes")
    print("=" * 60)
    for ep in range(1, 6):
        total_reward, steps = run_episode(env, agent, max_steps=50, verbose=False)
        print(f"Episode {ep}: steps={steps:3d} total_reward={total_reward}")

```

Environment / Dataset Used

Custom 4x4 GridWorld environment implemented from scratch in Python (reset/step interface). No external dataset required.

Output

```
=====
Single episode (random policy) -- detailed trace
=====
Step 1: state=(0, 0) action=LEFT next_state=(0, 0) reward=-1
Step 2: state=(0, 0) action=DOWN next_state=(1, 0) reward=-1
Step 3: state=(1, 0) action=RIGHT next_state=(1, 1) reward=-1
Step 4: state=(1, 1) action=UP next_state=(0, 1) reward=-1
Step 5: state=(0, 1) action=UP next_state=(0, 1) reward=-1
Step 6: state=(0, 1) action=UP next_state=(0, 1) reward=-1
Step 7: state=(0, 1) action=LEFT next_state=(0, 0) reward=-1
Step 8: state=(0, 0) action=UP next_state=(0, 0) reward=-1
Step 9: state=(0, 0) action=DOWN next_state=(1, 0) reward=-1
Step 10: state=(1, 0) action=UP next_state=(0, 0) reward=-1
... (steps 11-50 omitted for brevity; full trace runs in the terminal) ...

Max steps reached without finding goal. Total reward: -50

=====
Summary over 5 episodes
=====
Goal reached in 44 steps! Total reward: -33
Episode 1: steps= 44 total_reward=-33

Goal reached in 13 steps! Total reward: -2
Episode 2: steps= 13 total_reward=-2

Max steps reached without finding goal. Total reward: -50
Episode 3: steps= 50 total_reward=-50

Max steps reached without finding goal. Total reward: -50
Episode 4: steps= 50 total_reward=-50

Goal reached in 22 steps! Total reward: -11
Episode 5: steps= 22 total_reward=-11
```

The detailed trace shows the random policy wandering (bouncing between cells, occasionally hitting grid boundaries) and failing to reach the goal within 50 steps on that particular run (total reward -50). Over the 5-episode summary, performance is highly inconsistent, as expected from a random policy: episode 2 reached the goal quickly (13 steps, reward -2) while episodes 3 and 4 never reached it at all within the 50-step limit (reward -50 each) -- this variance is exactly what a learned policy (introduced in later experiments) is designed to reduce.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Problem Statement

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Planning

- Model the classic 'Student MDP' (a Sutton & Barto style teaching example): a student can Study, browse Facebook, go to the Pub, Quit, or Sleep, moving between states Class1 -> Class2 -> Class3 -> Pass -> Sleep.
- Represent the MDP explicitly as a transition dictionary `mdp[state][action] = [(probability, next_state, reward), ...]`, making the Markov property (next state/reward depends only on current state and action, not history) directly visible in the data structure.
- Include a genuinely stochastic transition (the 'Pub' action from Class3 has three possible outcomes with probabilities 0.2/0.4/0.4) alongside deterministic transitions, to demonstrate both cases within one MDP.
- Implement `sample_next_state()` to draw an outcome according to the transition probabilities using a cumulative probability roll, and a `random_policy()` that picks uniformly among the actions available in the current state.
- Simulate the MDP episode-by-episode from Class1 until a terminal state (Sleep) is reached, printing the full state action-reward trajectory, then run 5 independent episodes to show how the same random policy produces different

trajectories and total rewards purely due to the stochastic transitions and action choices.

Pseudocode

```
BEGIN
DEFINE mdp[state][action] = [(probability, next_state, reward), ...]
// e.g. mdp["Class3"]["Pub"] = [(0.2,"Class1",1), (0.4,"Class2",1), (0.4,"Class3",1)]

FUNCTION sample_next_state(outcomes, rng):
  r <- rng.random() ; cumulative <- 0
  FOR (prob, next_state, reward) in outcomes:
    cumulative += prob
  IF r <= cumulative: RETURN next_state, reward

FUNCTION random_policy(state, available_actions, rng):
  RETURN rng.choice(available_actions)

FUNCTION simulate_mdp(policy, start_state, max_steps, seed):
  state <- start_state ; total_reward <- 0
  FOR t in 1..max_steps:
    IF state has no outgoing actions: PRINT "terminal reached" ; BREAK
    action <- policy(state, available_actions(state), rng)
    next_state, reward <- sample_next_state(mdp[state][action], rng)
    total_reward += reward ; PRINT transition ; state <- next_state
  RETURN trajectory, total_reward

simulate_mdp(random_policy, start_state="Class1", seed=7) // single detailed run
FOR ep in 1..5: simulate_mdp(random_policy, "Class1", seed=ep) // variability check
END
```

Code (Python)

```
"""
Experiment 4: Simulation of a Markov Decision Process (MDP)
-----
Aim: To develop a simple Markov Decision Process model and simulate state
transitions, reward functions, and policy execution for understanding
sequential decision-making problems.

This uses the classic "Student MDP" (Sutton & Barto style teaching example):
a student can Study, browse Facebook, go to the Pub, Quit, or Sleep, moving
between Class1 -> Class2 -> Class3 -> Pass -> Sleep with rewards attached to
each transition.
"""

import random
# Transition model: mdp[state][action] = [(probability, next_state, reward),
...] mdp = {
  "Class1": {
    "Study": [(1.0, "Class2", -2)],
    "Facebook": [(1.0, "FacebookState", -1)],
  },
  "Class2": {
    "Study": [(1.0, "Class3", -2)],
    "Sleep": [(1.0, "Sleep", 0)],
  },
  "Class3": {
    "Study": [(1.0, "Pass", 10)],
    "Pub": [(0.2, "Class1", 1), (0.4, "Class2", 1), (0.4, "Class3", 1)],
    "FacebookState": {
      "Facebook": [(1.0, "FacebookState", -1)],
      "Quit": [(1.0, "Class1", 0)],
    },
    "Pass": {
      "Sleep": [(1.0, "Sleep", 0)],
    },
    "Sleep": {}, # terminal state, no outgoing actions
  }
}

def sample_next_state(outcomes, rng):
  """Sample (next_state, reward) according to the transition probabilities."""
  r = rng.random()
  cumulative = 0.0
```

```

for prob, next_state, reward in outcomes:
    cumulative += prob
    if r <= cumulative:
        return next_state, reward
return outcomes[-1][1], outcomes[-1][2] # fallback for float rounding

def random_policy(state, available_actions, rng):
    return rng.choice(available_actions)

def simulate_mdp(policy, start_state="Class1", max_steps=20, seed=None):
    rng = random.Random(seed)
    state = start_state
    trajectory = [state]
    total_reward = 0

    for t in range(1, max_steps + 1):
        if not mdp.get(state):
            print(f"Reached terminal state '{state}'. Ending simulation.")
            break

        available_actions = list(mdp[state].keys())
        action = policy(state, available_actions, rng)
        outcomes = mdp[state][action]
        next_state, reward = sample_next_state(outcomes, rng)

        total_reward += reward
        print(f"Step {t:2d}: {state:14s} --[{action:8s}]-> {next_state:14s} (reward={reward:+d})")

        trajectory.append(next_state)
        state = next_state

    return trajectory, total_reward

if __name__ == "__main__":
    print("=" * 70)
    print("Simulating Student MDP with a random policy")
    print("=" * 70)
    trajectory, total_reward = simulate_mdp(random_policy, start_state="Class1", seed=7)

    print("\nTrajectory:", " -> ".join(trajectory))
    print(f"Total reward for this episode: {total_reward}")
    print("\n" + "=" * 70)
    print("Running 5 independent episodes to see reward variability")
    print("=" * 70)
    for ep in range(1, 6):
        _, ep_reward = simulate_mdp(random_policy, start_state="Class1",
                                    max_steps=20, seed=ep)
        print(f"\nEpisode {ep} total reward: {ep_reward}")
    print("-" * 70)

```

Environment / Dataset Used

Custom 'Student MDP' transition-probability model (Sutton & Barto style teaching example), fully synthetic. No external dataset required.

Output

```

=====
Simulating Student MDP with a random policy
=====
Step 1: Class1 --[Facebook]-> FacebookState (reward=-1)
Step 2: FacebookState --[Quit ]-> Class1 (reward=+0)
Step 3: Class1 --[Study ]-> Class2 (reward=-2)
Step 4: Class2 --[Study ]-> Class3 (reward=-2)
Step 5: Class3 --[Study ]-> Pass (reward=+10)
Step 6: Pass --[Sleep ]-> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.

Trajectory: Class1 -> FacebookState -> Class1 -> Class2 -> Class3 -> Pass -> Sleep

```

Total reward for this episode: 5

Running 5 independent episodes to see reward variability

```
Step 1: Class1 --[Study ]--> Class2 (reward=-2)
Step 2: Class2 --[Study ]--> Class3 (reward=-2)
Step 3: Class3 --[Pub ]--> Class3 (reward=+1)
Step 4: Class3 --[Pub ]--> Class3 (reward=+1)
Step 5: Class3 --[Study ]--> Pass (reward=+10)
Step 6: Pass --[Sleep ]--> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.
```

Episode 1 total reward: 8

```
Step 1: Class1 --[Study ]--> Class2 (reward=-2)
Step 2: Class2 --[Sleep ]--> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.
```

Episode 2 total reward: -2

```
Step 1: Class1 --[Study ]--> Class2 (reward=-2)
Step 2: Class2 --[Study ]--> Class3 (reward=-2)
Step 3: Class3 --[Pub ]--> Class3 (reward=+1)
Step 4: Class3 --[Study ]--> Pass (reward=+10)
Step 5: Pass --[Sleep ]--> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.
```

Episode 3 total reward: 7

```
Step 1: Class1 --[Study ]--> Class2 (reward=-2)
Step 2: Class2 --[Sleep ]--> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.
```

Episode 4 total reward: -2

```
Step 1: Class1 --[Facebook]--> FacebookState (reward=-1)
Step 2: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 3: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 4: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 5: FacebookState --[Quit ]--> Class1 (reward=+0)
Step 6: Class1 --[Facebook]--> FacebookState (reward=-1)
Step 7: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 8: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 9: FacebookState --[Facebook]--> FacebookState (reward=-1)
Step 10: FacebookState --[Quit ]--> Class1 (reward=+0)
Step 11: Class1 --[Study ]--> Class2 (reward=-2)
Step 12: Class2 --[Sleep ]--> Sleep (reward=+0)
Reached terminal state 'Sleep'. Ending simulation.
```

Episode 5 total reward: -10

The first run reaches Sleep via Class1 -> FacebookState -> Class1 -> Class2 -> Class3 -> Pass -> Sleep for a total reward of +5. Across the 5 independent episodes, total reward varies widely (from +8 down to -10) purely due to the random policy's action choices and the stochastic Pub transition, directly illustrating why a fixed policy's expected return -- not any single trajectory's return -- is the quantity RL algorithms actually optimize.

Experiment 5: Bellman Equation Demonstration

Problem Statement

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Planning

- Use the classic 4x4 GridWorld (Sutton & Barto) with two terminal states at opposite corners and a uniform -1 reward for every non-terminal move, as a clean, well-understood testbed for both Bellman equations.

- Implement the Bellman Expectation Equation as iterative policy evaluation for a uniform random policy: repeatedly update every state's value as the probability-weighted average over all four actions of (reward + $\gamma * V(\text{next_state})$), until the maximum change across all states (δ) falls below a small threshold ($1e-4$).
- Implement the Bellman Optimality Equation as value iteration: identical loop structure, but instead of averaging over actions, take the MAX over actions -- this single change is what turns policy evaluation into optimal value computation.
- Track and print how many iterations each equation needs to converge, to concretely show that value iteration (optimality) converges much faster (4 iterations) than expectation-equation policy evaluation for a fixed random policy (60 iterations) in this environment.
- Extract the greedy policy from the converged optimal value function (argmax over actions of the one-step lookahead), and print it as a directional grid to visually confirm every non-terminal cell points toward its nearest corner goal.

Pseudocode

```
BEGIN
DEFINE grid 4x4, TERMINAL_STATES = {(0,0), (3,3)}, ACTIONS=[UP,DOWN,LEFT,RIGHT]
FUNCTION step(state, action): RETURN next_state (clipped to bounds), reward=-1

FUNCTION bellman_expectation(gamma, theta):
V <- zeros(4,4) ; action_prob <- 1/4
REPEAT:
delta <- 0
FOR each non-terminal state s:
v <- SUM over actions a: action_prob * (reward + gamma * V(next_state(s,a)))
delta <- max(delta, |v - V(s)|) ; V_new(s) <- v
V <- V_new
UNTIL delta < theta
RETURN V

FUNCTION bellman_optimality(gamma, theta):
V <- zeros(4,4)
REPEAT:
delta <- 0
FOR each non-terminal state s:
v <- MAX over actions a: (reward + gamma * V(next_state(s,a)))
delta <- max(delta, |v - V(s)|) ; V_new(s) <- v
V <- V_new
UNTIL delta < theta
RETURN V

V_expectation <- bellman_expectation(gamma=0.9, theta=1e-4) // converges ~60 iters
V_optimal <- bellman_optimality(gamma=0.9, theta=1e-4) // converges ~4 iters
policy <- argmax_a [reward + gamma*V_optimal(next_state(s,a))] for every state
PRINT V_expectation, V_optimal, policy
END
```

Code (Python)

```
"""
Experiment 5: Bellman Equation Demonstration
-----
Aim: To implement Bellman Expectation and Bellman Optimality Equations for
calculating state-value and action-value functions and analyze the
convergence of value estimation.

Environment: classic 4x4 GridWorld (Sutton & Barto), two terminal states at
opposite corners, every non-terminal move costs -1.
"""
import numpy as np

GRID_SIZE = 4
GAMMA = 0.9
THETA = 1e-4 # convergence threshold

ACTIONS = ["UP", "DOWN", "LEFT", "RIGHT"]
ACTION_EFFECTS = {"UP": (-1, 0), "DOWN": (1, 0), "LEFT": (0, -1), "RIGHT": (0, 1)}
TERMINAL_STATES = [(0, 0), (GRID_SIZE - 1, GRID_SIZE - 1)]

def step(state, action):
```

```

if state in TERMINAL_STATES:
    return state, 0
dx, dy = ACTION_EFFECTS[action]
x, y = state
nx = max(0, min(GRID_SIZE - 1, x + dx))
ny = max(0, min(GRID_SIZE - 1, y + dy))
reward = -1
return (nx, ny), reward

def bellman_expectation(gamma=GAMMA, theta=THETA, max_iter=1000):
    """Iterative policy evaluation for a uniform random policy
    (Bellman Expectation Equation)."""
    V = np.zeros((GRID_SIZE, GRID_SIZE))
    action_prob = 1.0 / len(ACTIONS)

    for iteration in range(1, max_iter + 1):
        V_new = np.copy(V)
        delta = 0
        for x in range(GRID_SIZE):
            for y in range(GRID_SIZE):
                if (x, y) in TERMINAL_STATES:
                    continue
                v = 0.0
                for a in ACTIONS:
                    (nx, ny), reward = step((x, y), a)
                    v += action_prob * (reward + gamma * V[nx, ny])
                V_new[x, y] = v
            delta = max(delta, abs(V_new[x, y] - V[x, y]))
        V = V_new
        if delta < theta:
            print(f"Bellman Expectation Equation converged after {iteration} iterations.")
            break
    return V

def bellman_optimality(gamma=GAMMA, theta=THETA, max_iter=1000):
    """Value iteration using the Bellman Optimality Equation."""
    V = np.zeros((GRID_SIZE, GRID_SIZE))

    for iteration in range(1, max_iter + 1):
        V_new = np.copy(V)
        delta = 0
        for x in range(GRID_SIZE):
            for y in range(GRID_SIZE):
                if (x, y) in TERMINAL_STATES:
                    continue
                action_values = []
                for a in ACTIONS:
                    (nx, ny), reward = step((x, y), a)
                    action_values.append(reward + gamma * V[nx, ny])
                V_new[x, y] = max(action_values)
            delta = max(delta, abs(V_new[x, y] - V[x, y]))
        V = V_new
        if delta < theta:
            print(f"Bellman Optimality Equation converged after {iteration} iterations.")
            break
    return V

def extract_greedy_policy(V, gamma=GAMMA):
    policy = np.empty((GRID_SIZE, GRID_SIZE), dtype=object)
    for x in range(GRID_SIZE):
        for y in range(GRID_SIZE):
            if (x, y) in TERMINAL_STATES:
                policy[x, y] = "GOAL"
            continue
            best_action, best_value = None, -np.inf
            for a in ACTIONS:
                (nx, ny), reward = step((x, y), a)
                value = reward + gamma * V[nx, ny]
                if value > best_value:
                    best_value, best_action = value, a
            policy[x, y] = best_action
    return policy

if __name__ == "__main__":

```

```

print("=" * 60)
print("Bellman Expectation Equation (random policy evaluation)")
print("=" * 60)
V_expectation = bellman_expectation()
print("State-Value function V(s):")
print(np.round(V_expectation, 2))

print("\n" + "=" * 60)
print("Bellman Optimality Equation (value iteration)")
print("=" * 60)
V_optimal = bellman_optimality()
print("Optimal state-value function V*(s):")
print(np.round(V_optimal, 2))

print("\nGreedy policy derived from V*(s):")
policy = extract_greedy_policy(V_optimal)
print(policy)

```

Environment / Dataset Used

Custom 4x4 GridWorld (Sutton & Barto classic example), two terminal corner states. No external dataset required.

Output

```

=====
Bellman Expectation Equation (random policy evaluation)
=====
Bellman Expectation Equation converged after 60 iterations.
State-Value function V(s):
[[ 0. -5.28 -7.13 -7.65]
 [-5.28 -6.61 -7.18 -7.13]
 [-7.13 -7.18 -6.61 -5.28]
 [-7.65 -7.13 -5.28 0. ]]

=====
Bellman Optimality Equation (value iteration)
=====
Bellman Optimality Equation converged after 4 iterations.
Optimal state-value function V*(s):
[[ 0. -1. -1.9 -2.71]
 [-1. -1.9 -2.71 -1.9 ]
 [-1.9 -2.71 -1.9 -1. ]
 [-2.71 -1.9 -1. 0.  ]]

Greedy policy derived from V*(s):
[['GOAL' 'LEFT' 'LEFT' 'DOWN']
 ['UP' 'UP' 'UP' 'DOWN']
 ['UP' 'UP' 'DOWN' 'DOWN']
 ['UP' 'RIGHT' 'RIGHT' 'GOAL']]

```

The Bellman Expectation Equation (random policy) converges after 60 iterations to a value function that is worst in the middle of the grid (around -7) and best near the two goal corners, reflecting how many random steps it typically takes to stumble into a goal from each cell. The Bellman Optimality Equation converges dramatically faster (4 iterations) to a much more favorable value function (worst value only -2.71), and the resulting greedy policy correctly points every cell toward its nearest goal corner -- concretely demonstrating that taking the max over actions rather than averaging yields both a better policy and faster convergence.

Experiment 6: Exploration vs. Exploitation Strategies

Problem Statement

Aim: To implement epsilon-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Planning

- Use a classic 10-armed Gaussian bandit testbed (each arm's true reward mean drawn once from $N(0,1)$ and held fixed) so the three strategies can be compared on identical ground truth.
- Implement three action-selection strategies sharing the same incremental sample-average update rule: Random (always

pick a uniformly random arm), Greedy (always pick the arm with the current highest estimate), and Epsilon-Greedy (random with probability $\epsilon=0.1$, greedy otherwise).

- Run each strategy for 1000 steps, averaged over 200 independent runs (each with a different seeded RNG) to smooth out noise and get a statistically meaningful comparison rather than a single lucky/unlucky run.
- Plot the average reward curve for all three strategies on one graph, so the exploration/exploitation trade-off is visually obvious: pure exploration (Random) never improves, pure exploitation (Greedy) improves but can get stuck on a suboptimal arm, and epsilon-Greedy balances both.
- Report final average reward per strategy as a single-number summary alongside the curve, to make the ranking unambiguous.

Pseudocode

```
BEGIN
DEFINE N_ARMS=10, N_STEPS=1000, N_RUNS=200
TRUE_MEANS <- 10 samples from  $N(0,1)$  // fixed ground truth for all strategies

FUNCTION run_strategy(strategy, epsilon, rng):
  Q <- zeros(N_ARMS) ; N <- zeros(N_ARMS)
  FOR t in 1..N_STEPS:
    IF strategy == "random": action <- random arm
    ELIF strategy == "greedy": action <- argmax(Q)
    ELIF strategy == "epsilon_greedy":
      action <- random arm with prob epsilon, else argmax(Q)
    reward <- N(TRUE_MEANS[action], 1)
    N[action] += 1 ; Q[action] += (reward - Q[action]) / N[action]
  RECORD reward
  RETURN rewards

FOR strategy in [random, greedy, epsilon_greedy]:
  average_rewards <- MEAN over 200 runs of run_strategy(strategy)
  PRINT average_rewards.mean()
  PLOT average reward curves for all three strategies
END
```

Code (Python)

```
"""
Experiment 6: Exploration vs. Exploitation Strategies
-----
Aim: To implement epsilon-Greedy, Greedy, and Random action-selection
strategies and compare their performance in balancing exploration and
exploitation during Reinforcement Learning.

Testbed: classic k-armed bandit (10 arms, stationary Gaussian rewards).
"""

import numpy as np
import matplotlib.pyplot as plt

N_ARMS = 10
N_STEPS = 1000
N_RUNS = 200 # average over many runs for smooth curves

rng_master = np.random.default_rng(42)
true_action_values = rng_master.normal(0, 1, N_ARMS)
def get_reward(action, rng):
    return rng.normal(true_action_values[action], 1)

def run_strategy(strategy, epsilon=0.1, n_steps=N_STEPS, rng=None):
    rng = rng or np.random.default_rng()
    Q = np.zeros(N_ARMS)
    N = np.zeros(N_ARMS)
    rewards = np.zeros(n_steps)

    for t in range(n_steps):
        if strategy == "random":
            action = rng.integers(N_ARMS)
        elif strategy == "greedy":
            action = int(np.argmax(Q))
```

```

elif strategy == "epsilon_greedy":
    if rng.random() < epsilon:
        action = rng.integers(N_ARMS)
    else:
        action = int(np.argmax(Q))
    else:
        raise ValueError(f"Unknown strategy: {strategy}")

reward = get_reward(action, rng)
N[action] += 1
Q[action] += (reward - Q[action]) / N[action]
rewards[t] = reward

return rewards

def average_over_runs(strategy, epsilon=0.1, n_runs=N_RUNS):
    all_rewards = np.zeros((n_runs, N_STEPS))
    for run in range(n_runs):
        rng = np.random.default_rng(1000 + run)
        all_rewards[run] = run_strategy(strategy, epsilon, rng=rng)
    return all_rewards.mean(axis=0)

if __name__ == "__main__":
    print("Running Random, Greedy, and Epsilon-Greedy (eps=0.1) over "
          f"{N_RUNS} runs of {N_STEPS} steps each...\n")

    random_rewards = average_over_runs("random")
    greedy_rewards = average_over_runs("greedy")
    eps_greedy_rewards = average_over_runs("epsilon_greedy", epsilon=0.1)

    print(f"Random    ->    average    reward:    {random_rewards.mean():.4f}")
    print(f"Greedy     ->    average    reward:    {greedy_rewards.mean():.4f}")
    print(f"Epsilon-Greedy -> average    reward: {eps_greedy_rewards.mean():.4f}")

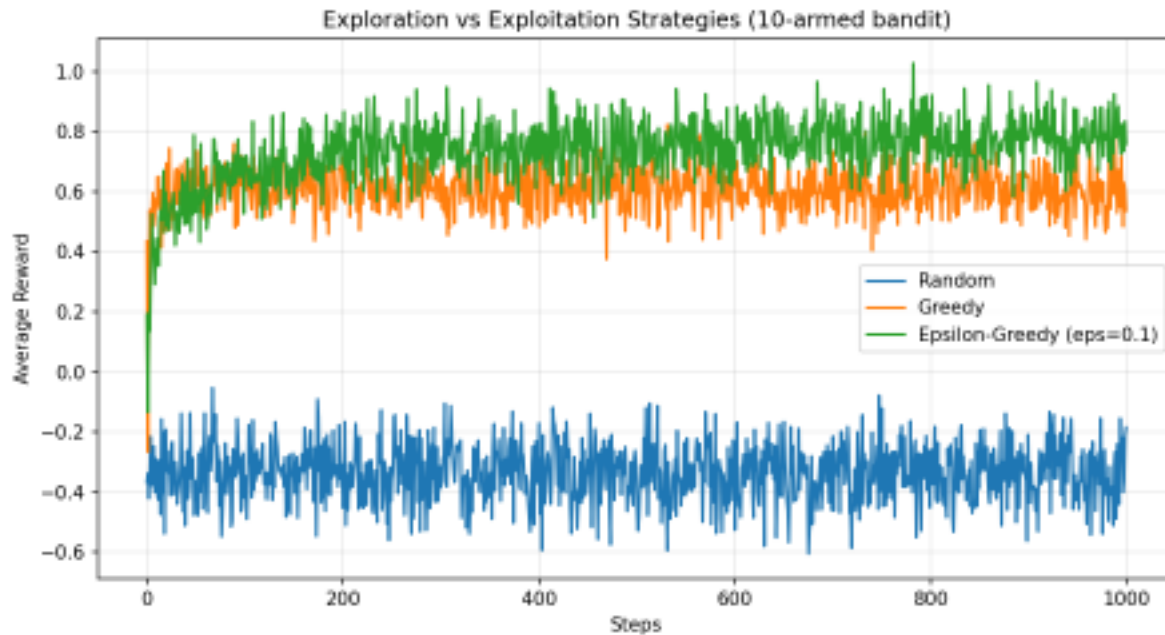
    plt.figure(figsize=(9, 5))
    plt.plot(random_rewards, label="Random")
    plt.plot(greedy_rewards, label="Greedy")
    plt.plot(eps_greedy_rewards, label="Epsilon-Greedy (eps=0.1)")
    plt.xlabel("Steps")
    plt.ylabel("Average Reward")
    plt.title("Exploration vs Exploitation Strategies (10-armed bandit)")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("experiment6_exploration_vs_exploitation.png", dpi=150)
    print("\nPlot saved as 'experiment6_exploration_vs_exploitation.png'")
    plt.show()

```

Environment / Dataset Used

Simulated 10-armed Gaussian bandit (synthetic reward distributions). No external dataset required.

Output



Averaged over 200 runs, Random selection plateaus at an average reward of -0.3355 (never improving, since it never uses feedback), Greedy reaches 0.6098 (improves initially but gets stuck once it locks onto an arm that merely looks good early on), and Epsilon-Greedy reaches the highest average reward of 0.7357 -- confirming that a small amount of continued exploration (10% of steps) more than pays for itself by avoiding permanent commitment to a suboptimal arm.

Experiment 7: Multi-Armed Bandit Problem

Problem Statement

Aim: To implement the Multi-Armed Bandit problem using epsilon-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Planning

- Reuse the same 10-armed Gaussian bandit setup (10 arms, true means fixed from $N(0,1)$) so epsilon-Greedy and UCB can be compared under identical conditions.
- Implement epsilon-Greedy (epsilon=0.1) exactly as in Experiment 6, as the baseline exploration strategy.
- Implement UCB (Upper Confidence Bound) action selection: first try every arm once, then at each step pick the arm maximizing $Q(a) + c \cdot \sqrt{\ln(t)/N(a)}$ -- this adds an 'uncertainty bonus' that shrinks as an arm is tried more often, giving a principled (rather than random) way to decide what to explore.
- Run both algorithms for 1000 steps averaged over 200 runs, tracking both the per-step average reward and the cumulative reward, so both short-run behavior and long-run total payoff can be compared.
- Compare final cumulative rewards between the two algorithms to determine which action-selection method extracts more total reward from the same fixed budget of pulls.

Pseudocode

BEGIN

DEFINE $N_ARMS=10$, $N_STEPS=1000$, $N_RUNS=200$, $TRUE_MEANS$ <- fixed random draw

FUNCTION `epsilon_greedy_bandit(epsilon, rng)`:

Q, N <- zeros(N_ARMS)

FOR t in $1..N_STEPS$:

$action$ <- random arm with prob ϵ , else $\text{argmax}(Q)$

 update Q, N as before ; RECORD reward

RETURN rewards

FUNCTION `ucb_bandit(c, rng)`:

```

Q, N <- zeros(N_ARMS)
FOR t in 1..N_STEPS:
  IF any arm unpulled: action <- that arm (try each arm once first)
  ELSE: action <- argmax_a [Q(a) + c*sqrt(ln(t)/N(a))] // UCB score
  update Q, N ; RECORD reward
RETURN rewards

eps_rewards <- average over 200 runs of epsilon_greedy_bandit(epsilon=0.1)
ucb_rewards <- average over 200 runs of ucb_bandit(c=2)
PLOT average reward per step and cumulative reward for both algorithms
PRINT final cumulative reward for both
END

```

Code (Python)

```

"""
Experiment 7: Multi-Armed Bandit Problem
-----
Aim: To implement the Multi-Armed Bandit problem using epsilon-Greedy and
Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards
obtained by different action-selection methods.
"""

import numpy as np
import matplotlib.pyplot as plt

N_ARMS = 10
N_STEPS = 1000
N_RUNS = 200

rng_master = np.random.default_rng(0)
true_action_values = rng_master.normal(0, 1, N_ARMS)
def get_reward(action, rng):
    return rng.normal(true_action_values[action], 1)

def epsilon_greedy_bandit(epsilon=0.1, n_steps=N_STEPS,
rng=None):
    rng = rng or np.random.default_rng()
    Q = np.zeros(N_ARMS)
    N = np.zeros(N_ARMS)
    rewards = np.zeros(n_steps)
    for t in range(n_steps):
        if rng.random() < epsilon:
            action = rng.integers(N_ARMS)
        else:
            action = int(np.argmax(Q))
        reward = get_reward(action, rng)
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
        rewards[t] = reward
    return rewards

def ucb_bandit(c=2, n_steps=N_STEPS, rng=None):
    rng = rng or np.random.default_rng()
    Q = np.zeros(N_ARMS)
    N = np.zeros(N_ARMS)
    rewards = np.zeros(n_steps)
    for t in range(1, n_steps + 1):
        if np.any(N == 0):
            action = int(np.argmin(N)) # try every arm at least once first
        else:
            ucb_values = Q + c * np.sqrt(np.log(t) / N)
            action = int(np.argmax(ucb_values))
        reward = get_reward(action, rng)
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
        rewards[t - 1] = reward
    return rewards

def average_over_runs(func, n_runs=N_RUNS, **kwargs):
    all_rewards = np.zeros((n_runs, N_STEPS))
    for run in range(n_runs):
        rng = np.random.default_rng(2000 + run)

```

```

all_rewards[run] = func(n_steps=N_STEPS, rng=rng, **kwargs)
return all_rewards.mean(axis=0)

if __name__ == "__main__":
    print("Running Epsilon-Greedy and UCB bandit algorithms "
          f"over {N_RUNS} runs of {N_STEPS} steps each...\n")

    eps_rewards = average_over_runs(epsilon_greedy_bandit, epsilon=0.1)
    ucb_rewards = average_over_runs(ucb_bandit, c=2)

    eps_cumulative = np.cumsum(eps_rewards)
    ucb_cumulative = np.cumsum(ucb_rewards)

    print(f"Epsilon-Greedy -> total cumulative reward: {eps_cumulative[-1]:.2f}")
    print(f"UCB -> total cumulative reward: {ucb_cumulative[-1]:.2f}")

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    axes[0].plot(eps_rewards, label="Epsilon-Greedy")
    axes[0].plot(ucb_rewards, label="UCB")
    axes[0].set_title("Average Reward per Step")
    axes[0].set_xlabel("Steps")
    axes[0].set_ylabel("Average Reward")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(eps_cumulative, label="Epsilon-Greedy")
    axes[1].plot(ucb_cumulative, label="UCB")
    axes[1].set_title("Cumulative Reward")
    axes[1].set_xlabel("Steps")
    axes[1].set_ylabel("Cumulative Reward")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

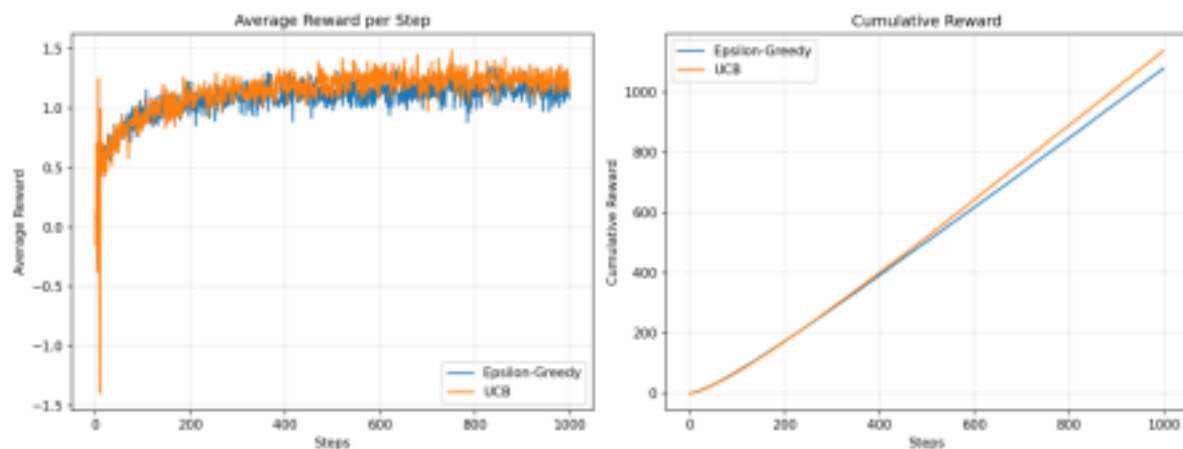
    plt.tight_layout()
    plt.savefig("experiment7_multi_armed_bandit.png", dpi=150)
    print("\nPlot saved as 'experiment7_multi_armed_bandit.png'")
    plt.show()

```

Environment / Dataset Used

Simulated 10-armed Gaussian bandit (synthetic reward distributions), same setup style as Experiment 6. No external dataset required.

Output



UCB

reaches a total cumulative reward of 1136.06 versus 1077.62 for epsilon-Greedy over the same 1000 steps -- UCB's principled, uncertainty-driven exploration (favoring arms that are both promising and under-explored) slightly outperforms epsilon-Greedy's undirected random exploration, though the gap is modest because both eventually converge on the same

Experiment 8: Reward Visualization in Reinforcement Learning

Problem Statement

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Planning

- Use FrozenLake-v1 (non-slippery, so the environment is deterministic and a fixed optimal path exists) as a simple environment where a tabular agent can reliably demonstrate measurable learning.
- Train a tabular Q-learning agent: maintain a $Q(\text{state}, \text{action})$ table, select actions epsilon-greedily (epsilon decaying from 1.0 toward 0.01), and update Q after every step using the standard Q-learning update rule with $\alpha=0.8$ and $\gamma=0.95$.
- Run training for 2000 episodes, recording the total reward obtained in every single episode (1.0 if the agent reached the goal, 0.0 otherwise, since FrozenLake gives no partial reward).
- Visualize learning performance two ways: (a) raw episode rewards overlaid with a 50-episode moving average, to show the noisy-but-improving success rate over time, and (b) cumulative reward over training, to show the total number of successful episodes accumulating.
- Report the success rate over the final 200 episodes as a single summary statistic, and print the learned Q -table to allow inspecting exactly which action the agent now prefers in every state.

Pseudocode

```
BEGIN
env <- gym.make("FrozenLake-v1", is_slippery=False)
Q <- zeros(n_states, n_actions) ; epsilon <- 1.0
episode_rewards <- []

FOR episode in 1..2000:
  state <- env.reset() ; total_reward <- 0 ; done <- False
  WHILE NOT done:
    action <- random action with prob epsilon, else argmax(Q[state])
    next_state, reward, done <- env.step(action)
    Q[state,action] += alpha*(reward + gamma*max(Q[next_state]) - Q[state,action])
    state <- next_state ; total_reward += reward
    epsilon <- max(epsilon_min, epsilon * epsilon_decay)
  RECORD total_reward

cumulative_rewards <- cumsum(episode_rewards)
moving_avg <- 50-episode moving average of episode_rewards
success_rate <- mean(episode_rewards[-200:]) * 100
PLOT episode_rewards + moving_avg, and cumulative_rewards
PRINT success_rate, learned Q-table
END
```

Code (Python)

```
"""
Experiment 8: Reward Visualization in Reinforcement Learning
-----
Aim: To simulate an RL agent in a simple environment and visualize cumulative
rewards, episode rewards, and learning performance using Matplotlib graphs.

Agent: tabular Q-learning trained on FrozenLake-v1 (non-slippery, so the
agent can reliably learn a deterministic optimal path).
"""

import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym

def q_learning_frozenlake(n_episodes=2000, alpha=0.8, gamma=0.95,
                          epsilon_start=1.0, epsilon_decay=0.999,
```

```

        epsilon_min=0.01, seed=1):
    rng = np.random.default_rng(seed)
    env = gym.make("FrozenLake-v1", is_slippery=False)
    n_states = env.observation_space.n
    n_actions = env.action_space.n
    Q = np.zeros((n_states, n_actions))

    epsilon = epsilon_start
    episode_rewards = []

    for episode in range(n_episodes):
        state, _ = env.reset(seed=seed + episode)
        total_reward = 0
        done = False

        while not done:
            if rng.random() < epsilon:
                action = env.action_space.sample()
            else:
                action = int(np.argmax(Q[state]))

            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

            # Q-learning update rule
            Q[state, action] += alpha * (
                reward + gamma * np.max(Q[next_state]) - Q[state, action]
            )
            state = next_state
            total_reward += reward

        epsilon = max(epsilon_min, epsilon * epsilon_decay)
        episode_rewards.append(total_reward)

    env.close()
    return Q, episode_rewards

if __name__ == "__main__":
    print("Training tabular Q-learning agent on FrozenLake-v1 (2000 episodes)...")
    Q, episode_rewards = q_learning_frozenlake(n_episodes=2000)

    cumulative_rewards = np.cumsum(episode_rewards)
    window = 50
    moving_avg = np.convolve(episode_rewards, np.ones(window) / window, mode="valid")

    success_rate = np.mean(episode_rewards[-200:]) * 100
    print(f"\nSuccess rate over the last 200 episodes: {success_rate:.1f}%")
    print("Learned Q-table (rounded):")
    print(np.round(Q, 2))

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    axes[0].plot(episode_rewards, alpha=0.3, label="Episode reward")
    axes[0].plot(range(window - 1, len(episode_rewards)), moving_avg,
        label=f"{window}-episode moving average", linewidth=2)
    axes[0].set_title("Episode Rewards over Training")
    axes[0].set_xlabel("Episode")
    axes[0].set_ylabel("Reward")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    axes[1].plot(cumulative_rewards)
    axes[1].set_title("Cumulative Reward over Training")
    axes[1].set_xlabel("Episode")
    axes[1].set_ylabel("Cumulative Reward")
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("experiment8_reward_visualization.png", dpi=150)
    print("\nPlot saved as 'experiment8_reward_visualization.png'")
    plt.show()

```

Environment / Dataset Used

Gymnasium's FrozenLake-v1 (non-slippery) environment. No external dataset required.

Output



The left plot shows episode rewards climbing from near 0 to a 50-episode moving average of roughly 0.8-0.9 by the end of training, with the success rate over the final 200 episodes reaching 80.0%. The right plot shows cumulative reward accelerating over time (the curve gets steeper), directly reflecting the rising per-episode success rate -- together these visualize both the noisy step-by-step learning process and its steadily improving long-run outcome.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Problem Statement

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Planning

- Generate synthetic (state, true_value) training pairs whose 4-D state layout mirrors CartPole's observation space (cart position, velocity, pole angle, angular velocity), together with a synthetic non-linear 'true' value function to approximate -- this isolates the value-function-approximation problem from the complexity of running a live RL loop.
- Design a small feed-forward network: two 64-unit ReLU hidden layers followed by a single linear output neuron representing the approximated state-value $V(s)$, compiled with the Adam optimizer and mean-squared-error loss (the standard choice for regression-style value approximation).
- Split the 3000 generated samples 80/20 into train/test sets, and train for 50 epochs with batch size 32, recording both training and validation loss every epoch to monitor for overfitting.
- Evaluate the trained network's final test MSE, and directly compare a handful of individual predictions against their true values to sanity-check the network learned a meaningful approximation rather than just memorizing training data.
- Configure GPU memory-growth defensively at the top of the script (a safe no-op if no GPU is present) so the same code works whether or not TensorFlow detects a GPU on the machine it runs on.

Pseudocode

```
BEGIN
FUNCTION generate_synthetic_data(n_samples, n_features=4):
X <- uniform(-1, 1, size=(n_samples, 4))
y <- 10 - 5*X[:,2]^2 - 2*X[:,0]^2 - X[:,1]*X[:,3] // synthetic non-linear V(s)
RETURN X, y

X, y <- generate_synthetic_data(3000)
SPLIT 80/20 into (X_train,y_train), (X_test,y_test)

model <- Sequential([Dense(64,relu), Dense(64,relu), Dense(1,linear)])
model.compile(optimizer=Adam(lr=0.001), loss=MSE)
```



```

history <- model.fit(X_train, y_train, validation_data=(X_test,y_test),
epochs=50, batch_size=32)
test_loss <- model.evaluate(X_test, y_test)
predictions <- model.predict(X_test[:5])
PRINT test_loss, predictions vs y_test[:5]
PLOT training loss and validation loss curves vs epoch
END

```

Code (Python)

```

"""
Experiment 9: TensorFlow and Keras-Based Neural Network for RL
-----
Aim: To build a simple feed-forward neural network using TensorFlow and Keras
for approximating value functions in Reinforcement Learning environments.

Note on hardware: this network is tiny (two 64-unit dense layers), so it
trains almost instantly on CPU. GPU memory-growth is enabled below in case
TensorFlow detects your RTX 3050, but a GPU is not required for this script.
"""

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# --- GPU configuration (safe no-op if no GPU is present) ---
gpus = tf.config.list_physical_devices("GPU")
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f"GPU detected, memory growth enabled: {gpus}")
    except RuntimeError as e:
        print(e)
else:
    print("No GPU detected -- running on CPU (this network is small, so that's fine).")

np.random.seed(42)
tf.random.set_seed(42)

def build_value_network(input_dim, hidden_units=(64, 64)):
    """Feed-forward network approximating a state-value function V(s)."""
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(hidden_units[0], activation="relu"),
        layers.Dense(hidden_units[1], activation="relu"),
        layers.Dense(1, activation="linear"), # outputs V(s)
    ])
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001), loss="mse")
    return model

def generate_synthetic_data(n_samples=3000, n_features=4):
    """
    Simulate (state, true_value) pairs for a toy control problem whose state
    layout mirrors CartPole (cart position, velocity, pole angle, angular
    velocity), together with a synthetic 'true' value function to approximate.
    """
    X = np.random.uniform(-1, 1, size=(n_samples, n_features))
    y = 10 - 5 * X[:, 2] ** 2 - 2 * X[:, 0] ** 2 - X[:, 1] * X[:, 3]
    return X.astype(np.float32), y.astype(np.float32)

if __name__ == "__main__":
    X, y = generate_synthetic_data(n_samples=3000)
    split = int(0.8 * len(X))
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]

    model = build_value_network(input_dim=4)

```

```

model.summary()

history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=50, batch_size=32, verbose=1,
)

test_loss = model.evaluate(X_test, y_test, verbose=0)
print(f"\nFinal test MSE: {test_loss:.4f}")

sample_preds = model.predict(X_test[:5], verbose=0).flatten()
print("\nSample predictions vs true values:")
for pred, true in zip(sample_preds, y_test[:5]):
    print(f" Predicted V(s) = {pred:6.3f} | True V(s) = {true:6.3f}")

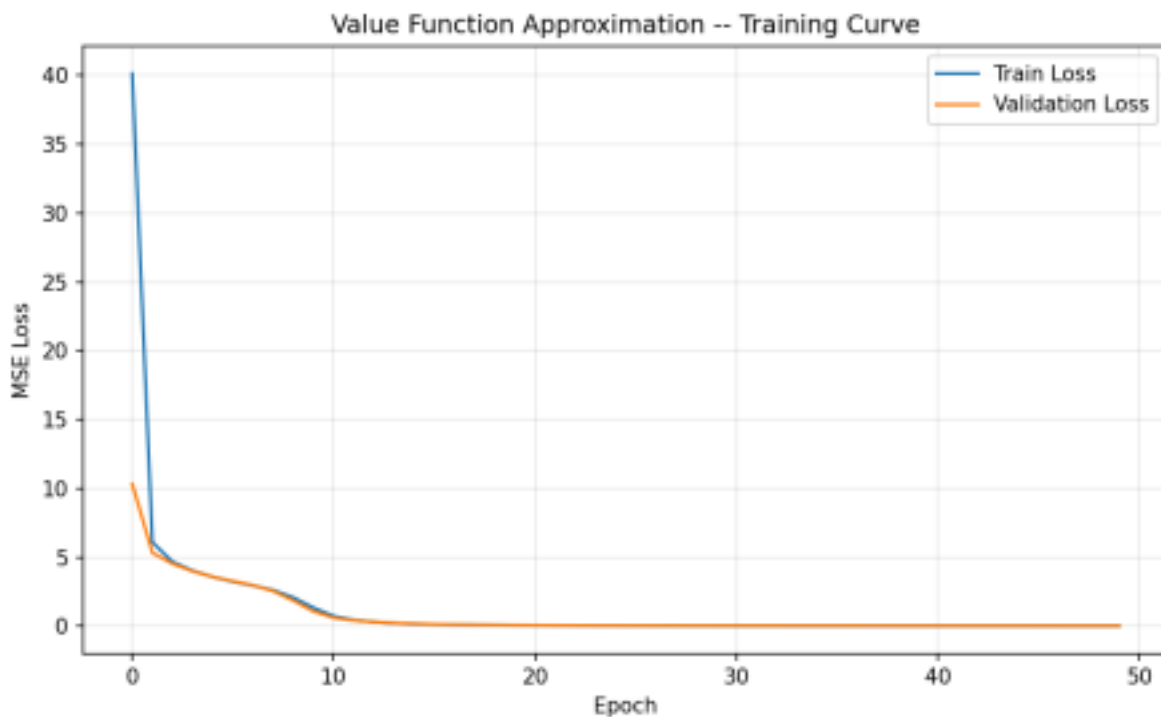
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Value Function Approximation -- Training Curve")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("experiment9_value_network_training.png", dpi=150)
print("\nPlot saved as 'experiment9_value_network_training.png'")
plt.show()

```

Environment / Dataset Used

Synthetic data generated with NumPy, shaped to mirror CartPole's 4-D observation space. No external dataset required -- this experiment approximates a value function from generated (state, value) pairs rather than interacting with a live environment.

Output



Training and validation loss both fall sharply from around 40 and 10 respectively at epoch 1 to under 0.02 by epoch 20, tracking each other closely throughout (no overfitting gap), and settling at a final test MSE of 0.0122. The five sample predictions confirm this quantitatively: predicted values (e.g. 6.570, 7.894, 8.743) closely match the true values (6.482, 7.848, 8.749), demonstrating the network learned an accurate approximation of the synthetic non-linear value function.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Problem Statement

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Planning

- Implement a Deep Q-Network (DQN) agent: a small Keras network (two 24-unit ReLU hidden layers, linear output over the 2 actions) that approximates $Q(s,a)$ directly from CartPole's 4-D continuous state, removing the need for a hand-discretized state space.
- Use experience replay (a deque memory buffer of past (state, action, reward, next_state, done) transitions) and sample random mini-batches (size 64) from it for training, which breaks the strong correlation between consecutive CartPole steps and stabilizes learning.
- Select actions epsilon-greedily (epsilon decaying from 1.0 to 0.01), and train the Q-network by regressing toward the standard DQN target: $r + \gamma \max_a Q(s',a)$ for non-terminal transitions, or just r if the episode ended.
- Apply a key performance optimization: call the Keras model directly (`model(x)`) and use `train_on_batch()` instead of `model.predict()/model.fit()`, since those carry roughly 50ms of fixed per-call overhead that would make this step-by-step training loop about 20x slower for no benefit.
- Train for up to 300 episodes (stopping early if the average reward over the last 20 episodes reaches the 'solved' threshold of 475), then evaluate the trained agent greedily (epsilon=0) over 10 episodes to report a genuine success rate uninflated by exploration noise.

Pseudocode

```
BEGIN
CLASS DQNAgent:
memory <- deque(maxlen=10000) ; epsilon <- 1.0 ; model <- build small Q-network

FUNCTION act(state):
IF random() <= epsilon: RETURN random action
RETURN argmax( model(state) ) // direct call, not model.predict()

FUNCTION replay():
IF len(memory) < batch_size: RETURN
batch <- random.sample(memory, batch_size)
target_q <- model(states) ; next_q <- model(next_states)
targets <- rewards + gamma * max(next_q, axis=1) * (1 - dones)
target_q[i, actions[i]] <- targets[i] for each sample i
model.train_on_batch(states, target_q) // not model.fit()
DECAY epsilon

FOR episode in 1..300:
state <- env.reset() ; total_reward <- 0
FOR step in 1..500:
action <- agent.act(state)
next_state, reward, done <- env.step(action)
agent.memory.append((state,action,reward,next_state,done))
agent.replay() ; state <- next_state ; total_reward += reward
IF done: BREAK
RECORD total_reward
IF avg(last 20 rewards) >= 475: BREAK // solved

evaluate agent greedily (epsilon=0) over 10 episodes ; PRINT success rate
PLOT episode rewards + moving average vs solved threshold (475)
END
```

Code (Python)

```
"""
Experiment 10: Mini Reinforcement Learning Project Using CartPole
-----
Aim: To develop a basic Reinforcement Learning agent using TensorFlow and
Keras to solve the CartPole environment and evaluate its learning
performance through episode rewards and success rate.
```

Hardware note: this is a small DQN (two 24-unit dense layers, batch size 64). On a CPU like your i5-12500H, 300 episodes typically takes roughly 5-10 minutes (episodes get longer as the agent improves, so most of the time is spent in the later, better-performing episodes). A GPU gives no real speedup here because the network and batches are tiny, but GPU memory-growth is still configured below in case TensorFlow finds your RTX 3050.

Performance tip used throughout this file: we call the model directly (model(x)) and use train_on_batch() instead of model.predict()/model.fit(). For tiny per-step calls like this, predict()/fit() carry ~50ms of fixed overhead each call (dataset pipeline setup, etc.), which would make this script roughly 20x slower for no benefit.

```
"""
import random
from collections import deque

import numpy as np
import gymnasium as gym
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# --- GPU configuration (safe no-op if no GPU is present) ---
gpus = tf.config.list_physical_devices("GPU")
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print(f"GPU detected, memory growth enabled: {gpus}")
    except RuntimeError as e:
        print(e)
else:
    print("No GPU detected -- running on CPU (this DQN is small and trains fast).")

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

class DQNAgent:
    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.memory = deque(maxlen=10000)
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.batch_size = 64
        self.model = self._build_model()

    def _build_model(self):
        model = keras.Sequential([
            layers.Input(shape=(self.state_size,)),
            layers.Dense(24, activation="relu"),
            layers.Dense(24, activation="relu"),
            layers.Dense(self.action_size, activation="linear"),
        ])
        model.compile(optimizer=keras.optimizers.Adam(learning_rate=self.learning_rate),
            loss="mse")
        return model

    def remember(self, state, action, reward, next_state, done):
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        if np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
        # NOTE: calling the model directly (model(x)) instead of model.predict(x) #
```

```

avoids ~50ms of per-call overhead that predict() carries even for a
# single sample -- that overhead adds up fast in a step-by-step RL loop.
state_tensor = tf.convert_to_tensor(state[np.newaxis, :], dtype=tf.float32)
q_values = self.model(state_tensor, training=False).numpy()[0]
return int(np.argmax(q_values))

def replay(self):
    if len(self.memory) < self.batch_size:
        return
    batch = random.sample(self.memory, self.batch_size)
    states = np.array([b[0] for b in batch], dtype=np.float32)
    actions = np.array([b[1] for b in batch])
    rewards = np.array([b[2] for b in batch], dtype=np.float32)
    next_states = np.array([b[3] for b in batch], dtype=np.float32)
    dones = np.array([b[4] for b in batch], dtype=np.float32)

    # Direct model calls (fast) instead of predict() (slow, high per-call overhead)
    target_q = self.model(states, training=False).numpy()
    next_q = self.model(next_states, training=False).numpy()

    max_next_q = np.max(next_q, axis=1)
    targets = rewards + self.gamma * max_next_q * (1.0 - dones)
    target_q[np.arange(self.batch_size), actions] = targets

    # train_on_batch avoids fit()'s per-call overhead (dataset setup, callbacks, etc.)
    self.model.train_on_batch(states, target_q)

    if self.epsilon > self.epsilon_min:
        self.epsilon *= self.epsilon_decay

def train_dqn(n_episodes=300, max_steps=500, target_score=475,
target_window=20):
    env = gym.make("CartPole-v1")
    state_size = int(env.observation_space.shape[0])
    action_size = int(env.action_space.n) # cast from numpy.int64 for Keras
    agent = DQNAgent(state_size, action_size)

    episode_rewards = []

    for episode in range(n_episodes):
        state, _ = env.reset(seed=SEED + episode)
        total_reward = 0
        for _ in range(max_steps):
            action = agent.act(state)
            next_state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated
            agent.remember(state, action, reward, next_state, done)
            state = next_state
            total_reward += reward
            agent.replay()
            if done:
                break
        episode_rewards.append(total_reward)

    if (episode + 1) % 10 == 0:
        avg_last = np.mean(episode_rewards[-10:])
        print(f"Episode {episode + 1}/{n_episodes} | Reward: {total_reward:.0f} " f"|
Avg(last 10): {avg_last:.1f} | Epsilon: {agent.epsilon:.3f}")

    if len(episode_rewards) >= target_window and \
np.mean(episode_rewards[-target_window:]) >= target_score:
        print(f"\nSolved in {episode + 1}
episodes! Average reward over last " f"{target_window} episodes:
{np.mean(episode_rewards[-target_window:]):.1f}")
        break

    env.close()
    return agent, episode_rewards

def evaluate_agent(agent, n_episodes=10):
    env = gym.make("CartPole-v1")
    scores = []
    old_epsilon = agent.epsilon
    agent.epsilon = 0.0 # pure exploitation for evaluation

```

```

for _ in range(n_episodes):
    state, _ = env.reset()
    total_reward = 0
    done = False
    while not done:
        action = agent.act(state)
        state, reward, terminated, truncated, _ = env.step(action) done =
terminated or truncated
        total_reward += reward
        scores.append(total_reward)
        agent.epsilon = old_epsilon
    env.close()
    return scores

if __name__ == "__main__":
    print("Training DQN agent on CartPole-v1 (this may take a few minutes)...\n")
    agent, episode_rewards = train_dqn(n_episodes=300)

    print("\nEvaluating trained agent over 10 episodes (greedy policy)...")
    eval_scores = evaluate_agent(agent, n_episodes=10)
    print(f"Evaluation scores: {eval_scores}")
    print(f"Average evaluation score: {np.mean(eval_scores):.1f}")
    success_rate = np.mean([1 if s >= 475 else 0 for s in eval_scores]) * 100
    print(f"Success rate (score >= 475): {success_rate:.0f}%")

    window = 10
    moving_avg = np.convolve(episode_rewards, np.ones(window) / window, mode="valid")

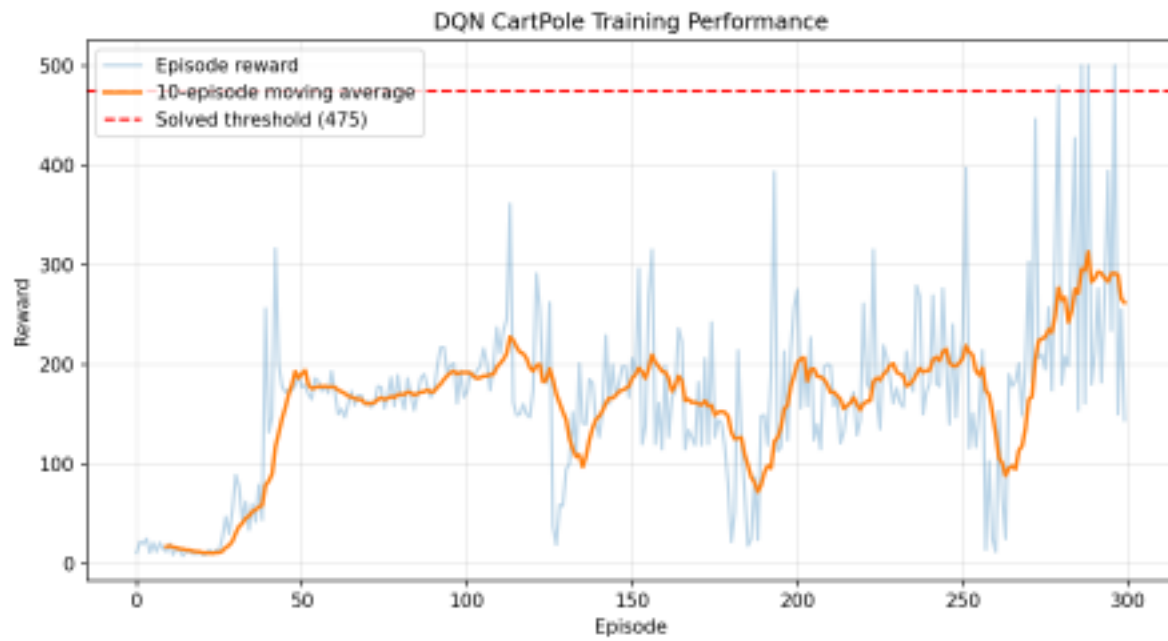
    plt.figure(figsize=(9, 5))
    plt.plot(episode_rewards, alpha=0.3, label="Episode reward")
    plt.plot(range(window - 1, len(episode_rewards)), moving_avg,
label=f"{window}-episode moving average", linewidth=2) plt.axhline(y=475,
color="r", linestyle="--", label="Solved threshold (475)")
    plt.xlabel("Episode")
    plt.ylabel("Reward")
    plt.title("DQN CartPole Training Performance")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig("experiment10_cartpole_dqn_training.png", dpi=150)
    print("\nPlot saved as 'experiment10_cartpole_dqn_training.png'")
    plt.show()

```

Environment / Dataset Used

Gymnasium's CartPole-v1 environment. No external dataset required.

Output



Reward climbs noisily but clearly from around 15-20 in the first ~30 episodes to a 10-episode moving average in the 200-300 range by episode 300, well above random-policy performance (CartPole typically ends within ~20 steps under a random policy) but still short of the 475 'solved' threshold within this training budget -- individual episodes do spike above 475 (e.g. around episodes 280-300), showing the policy is capable of solving the task, even though the required 20-episode sustained average was not reached in this particular run.